

SPI / I2C 통합 통신 HW 설계 및 검증

Singleboard & Two-board Basys3 기반 구현

서울 기술교육 센터 | 우상욱

목차 (Table of Contents)

01

서론 (Introduction)

프로젝트 개요 및 개발 목적

03

통신 모듈 설계

SPI 및 I2C 프로토콜 구현 상세

05

결론 (Conclusion)

Trouble Shooting 및 향후 개선 계획

02

전체 시스템 구조

Singleboard & Two-board 아키텍처

04

시스템 검증 (Verification)

UVM 기반 고도화 객체지향 검증 전략

1. 서론 (Introduction)

Project Overview & Objectives

프로젝트 개발 목적

프로토콜 구조 직접 구현

대표적 직렬 통신인 SPI와 I2C를 직접 구현하며 실질적으로 이해합니다.

Master/Slave 설계

데이터 전송 방향에 따른 Master와 Slave 각각의 동작 방식 및 FSM(Finite State Machine) 설계 방식을 학습합니다.

실보드 환경 연동 검증

단일보드 내부 Loopback 테스트와 물리적인 2개의 보드(Two-board)를 연결하는 환경의 차이를 경험하고 트러블슈팅합니다.

2. 전체 시스템 구조

System Architecture Targets

| 빌드 타겟 비교 (Single vs Multi-board)

Singleboard (통합 검증용)

- 하나의 FPGA 내부에 **Master와 Slave**를 모두 인스턴스화하여 외부 배선 없이 연결.
- 초기 기능 검증(Bring-up)에 가장 효율적인 구조.
- SPI Overwrite 및 Readback, I2C 통신 동작 즉각 확인.

Master_basys3

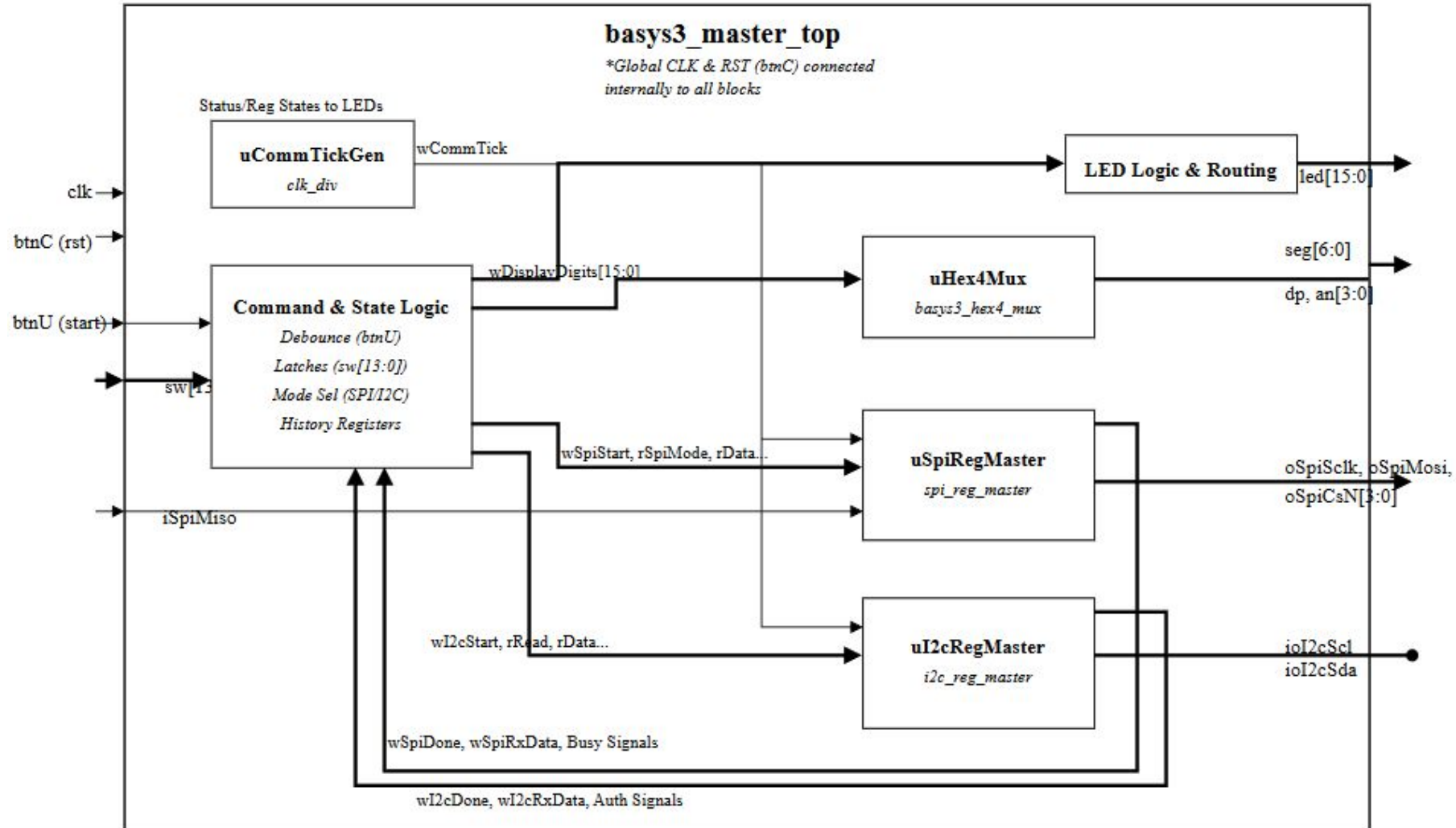
- 이중 보드 시스템에서 **Master 역할** 전담.
- SPI/I2C 통신 신호 생성 및 스위치/버튼을 통한 Transaction 트리거.

Slave_basys3

- Master의 요청을 수신하고 응답하는 **Slave 전용** 설계.
- 수신 데이터를 **Register Bank**에 저장 후 FND로 시각적 표시.

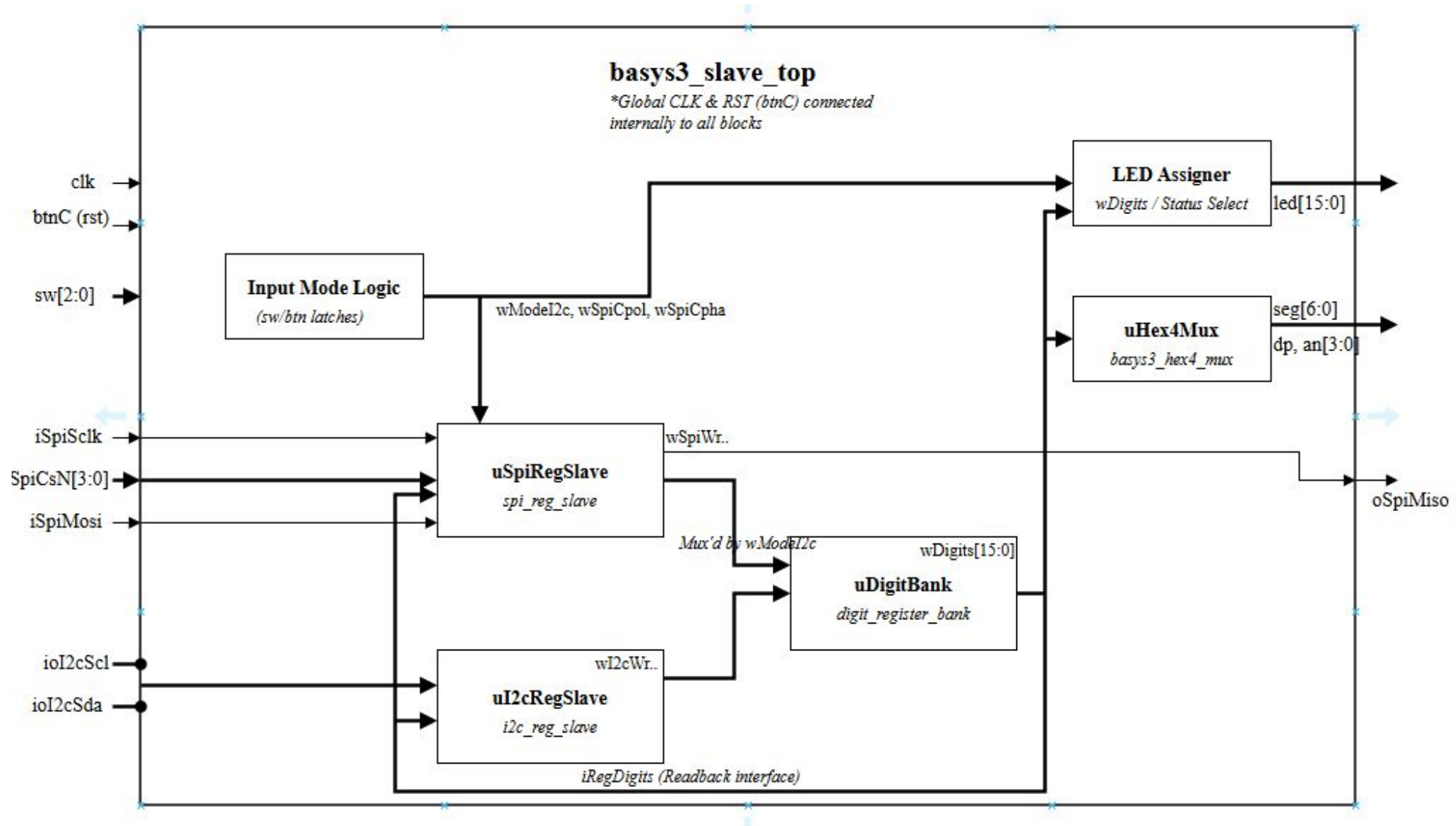
2.1 Top Diagram

Master_Top



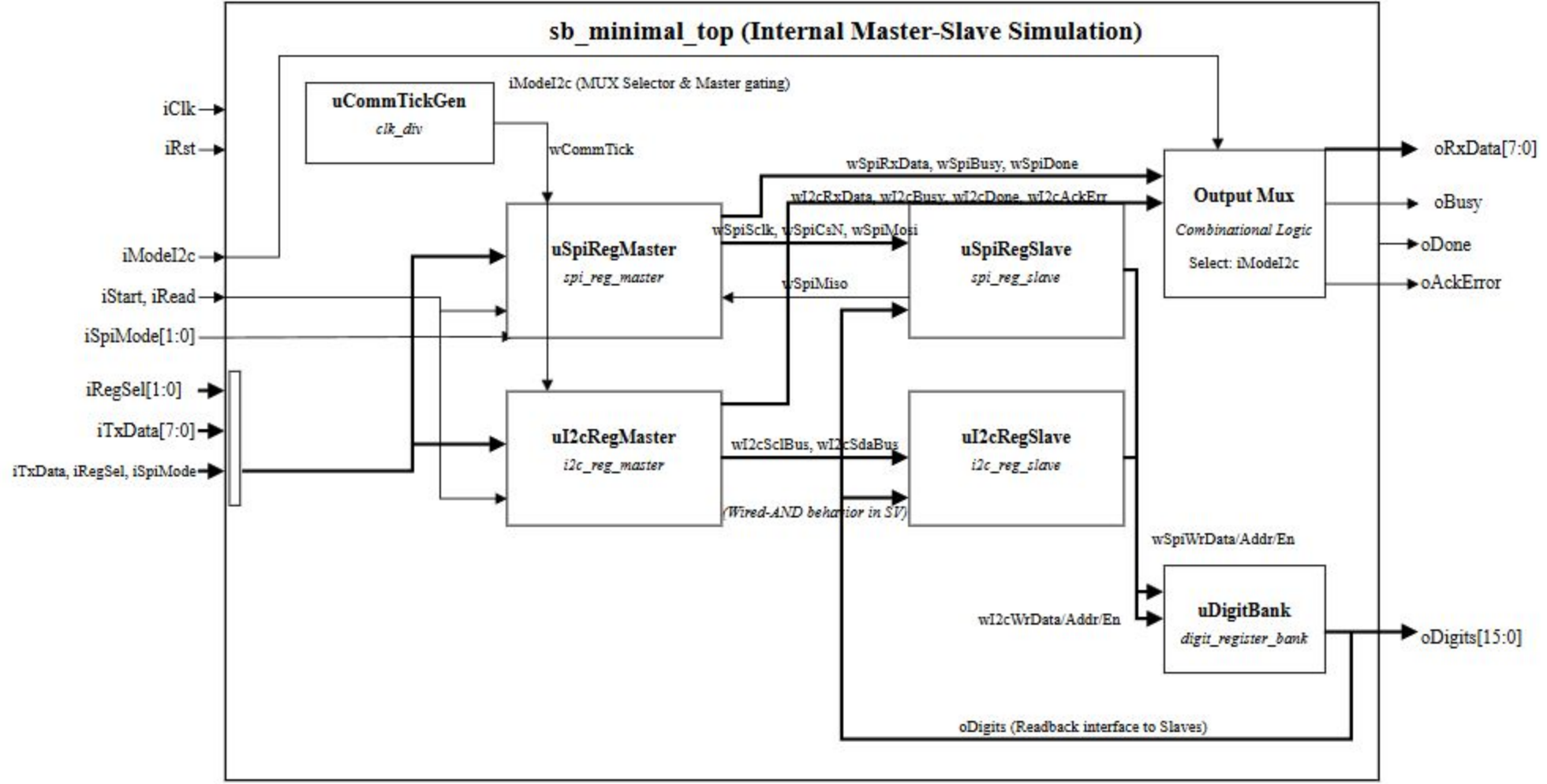
2.1 Top Diagram

Slave_Top



2.1 Top Diagram

SingleBoar



2.2 통신 시스템 비교

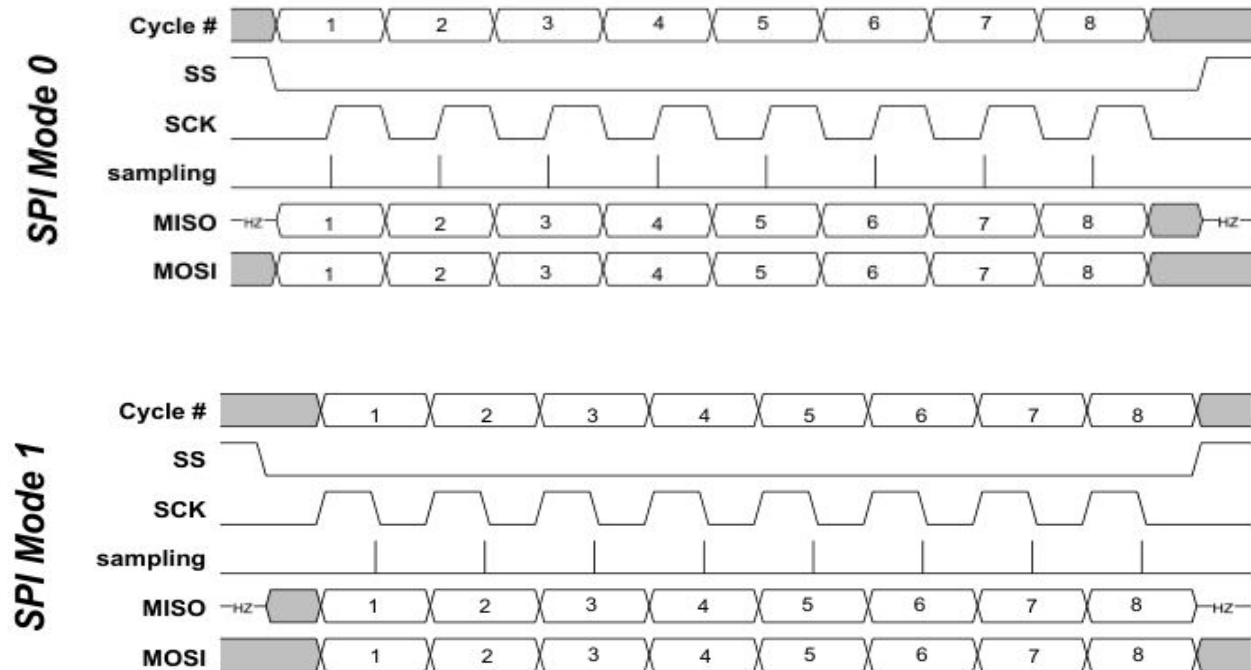
Compare UART, SPI, I2C

항목	UART	I2C	SPI
구조	1:1	공유 버스	CS 기반 선택
속도	낮음~중간	낮음~중간	높음
장치 수 확장	불리함	유리함	CS 수에 좌우
추천 상황	로그, 모듈 제어	센서 묶기	고속 데이터

3.1 통신 모듈 설계: SPI

Serial Peripheral Interface Design

- **동작 모드 지원:** 스위치를 통해 , CPOL, CPHA 조합 제어 가능.
- **Slave 대상 선택:** 스위치 입력을 통해 CS라인을 개별적으로 활성화하여 대상을 선택.



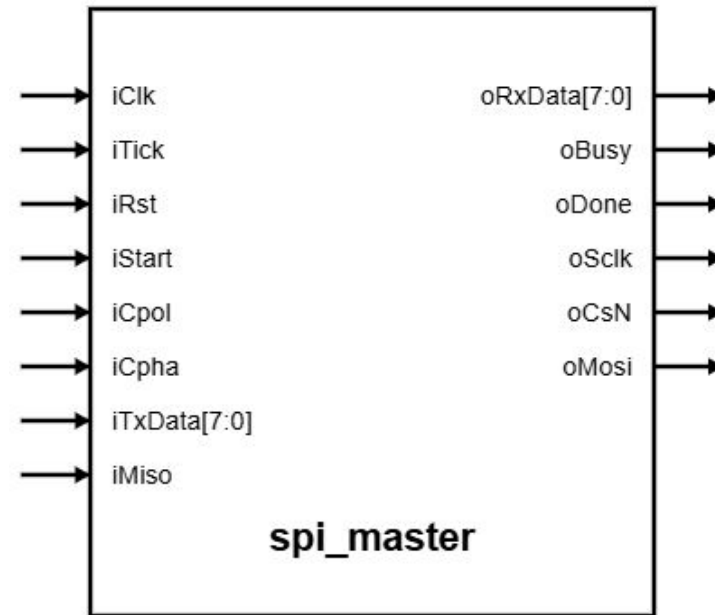
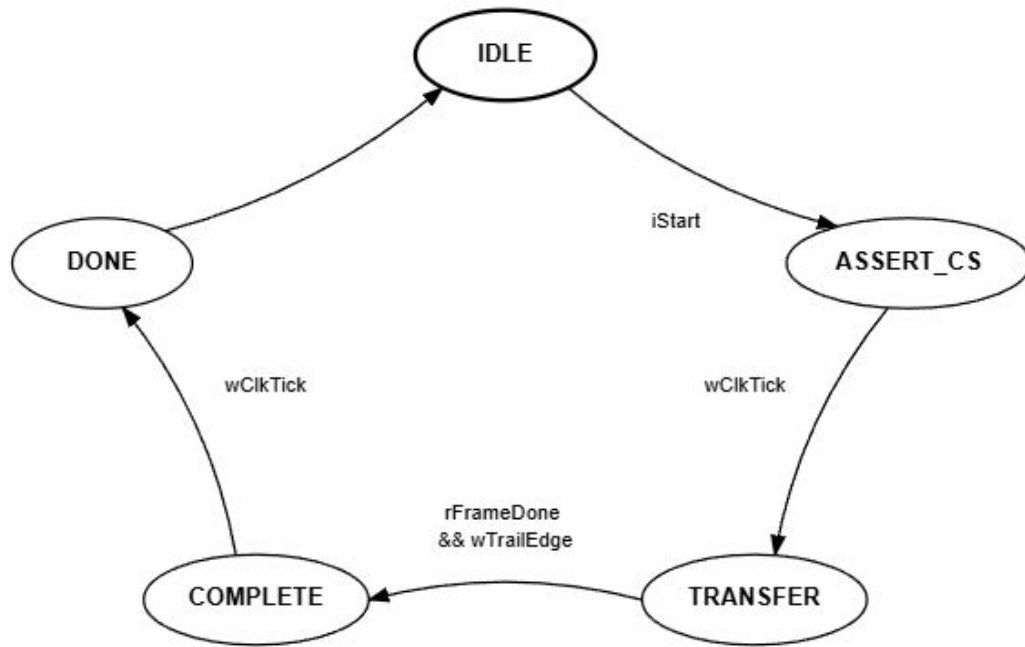
- Leading edge: Rising, sample
- Trailing edge: Falling, setup

- Leading edge: Rising, setup
- Trailing edge: Falling, sample

3.1 통신 모듈 설계: SPI_MASTER

Serial Peripheral Interface Design

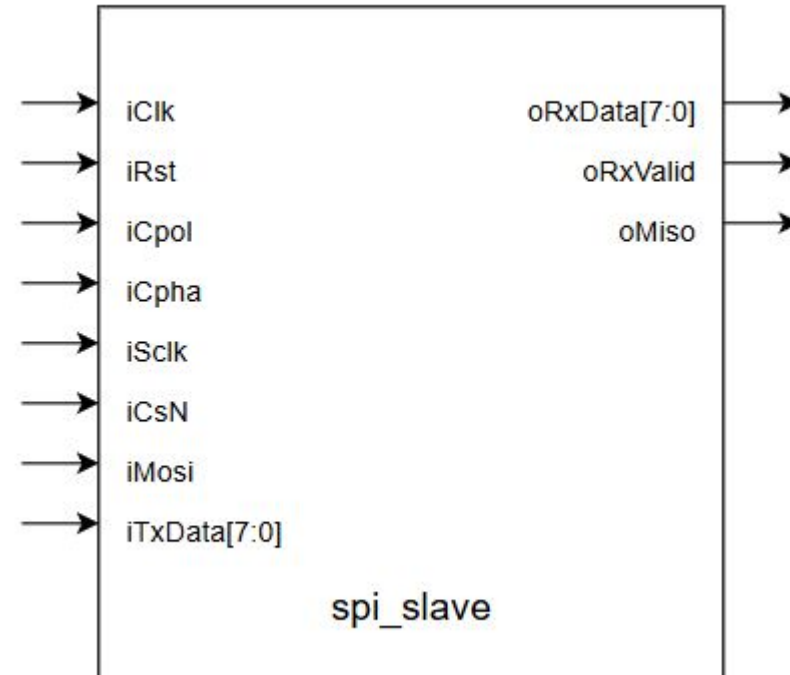
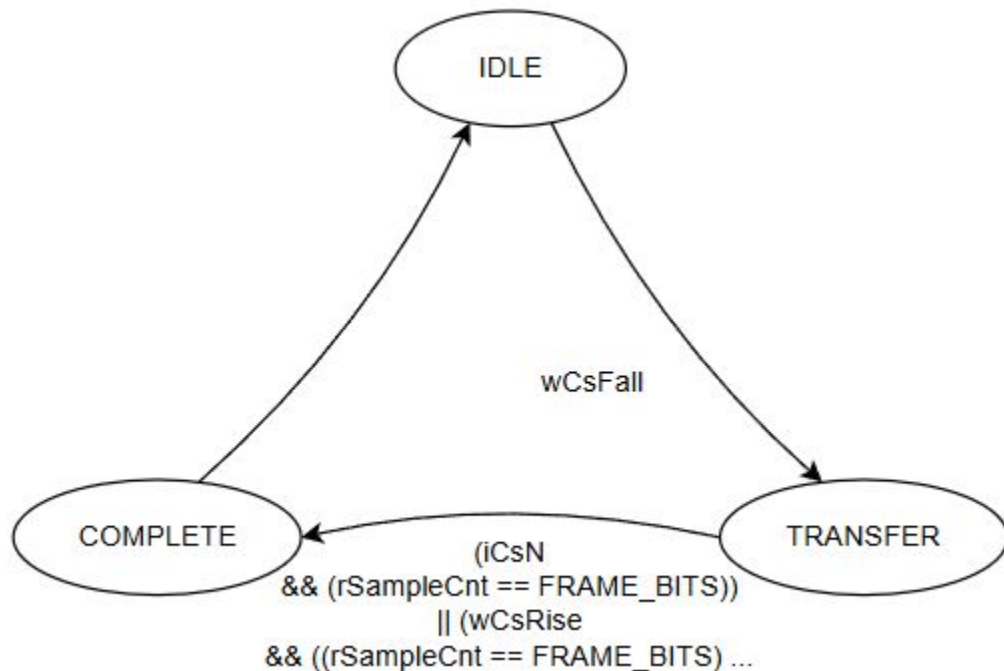
- **FSM 기반 제어:** State를 5개로 나누어 데이터 시트에 맞게 8bit data를 spi통신 프로토콜로 변환
- **Master 신호:** Sclk과 CS, Mosi 신호를 만들어서 slave에게 전달한다



3.1 통신 모듈 설계: SPI_SLAVE

Serial Peripheral Interface Design

- **FSM 기반 제어:** State를 3개로 나누어 데이터 시트에 맞게 mosi 신호를 data로
- **Slave 신호:** Master로 부터 받은 신호를 변환하여 8bit데이터를 만들고 reg bank에 저장한다.



3.1 Simulation: SPI_MASTER

Waveform Analysis (기능 확인용 Testbench)



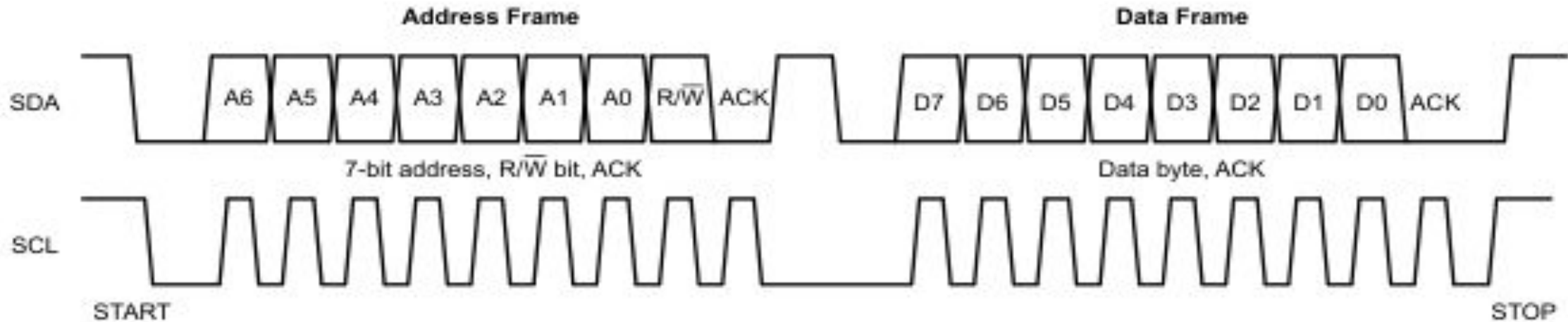
검증 포인트: SPI 시작/종료 확인, mode별 edge 타이밍 확인 데이터 Write 이후 내부 Register 갱신 및 기대값 일치 여부 점검.

Waveform Analysis (기능 확인용 Testbench)

검증 포인트: SPI 시작/종료 확인, mode별 edge 타이밍 확인 데이터 Write 이후 내부 **Register** 갱신 및 기대값 일치 여부 점검.

3.2 통신 모듈 설계: I2C

Inter-Integrated Circuit Design



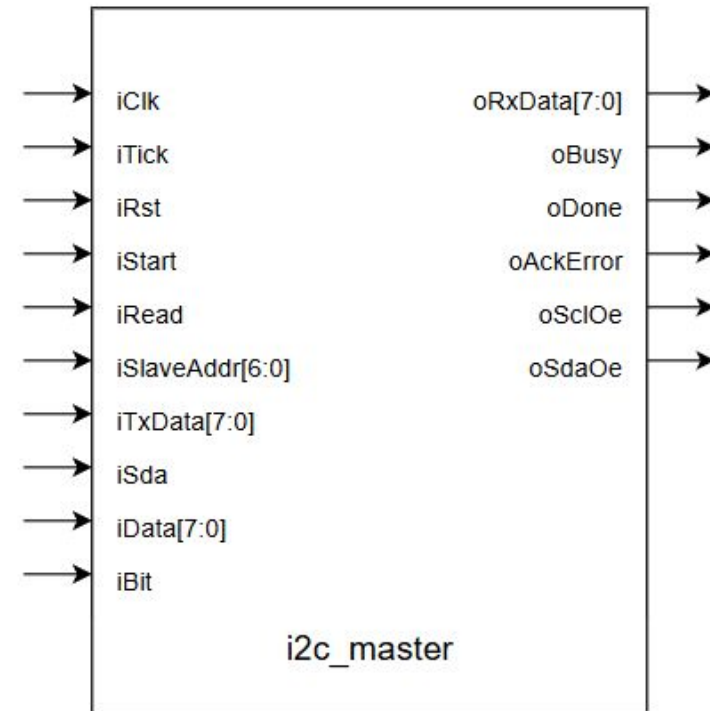
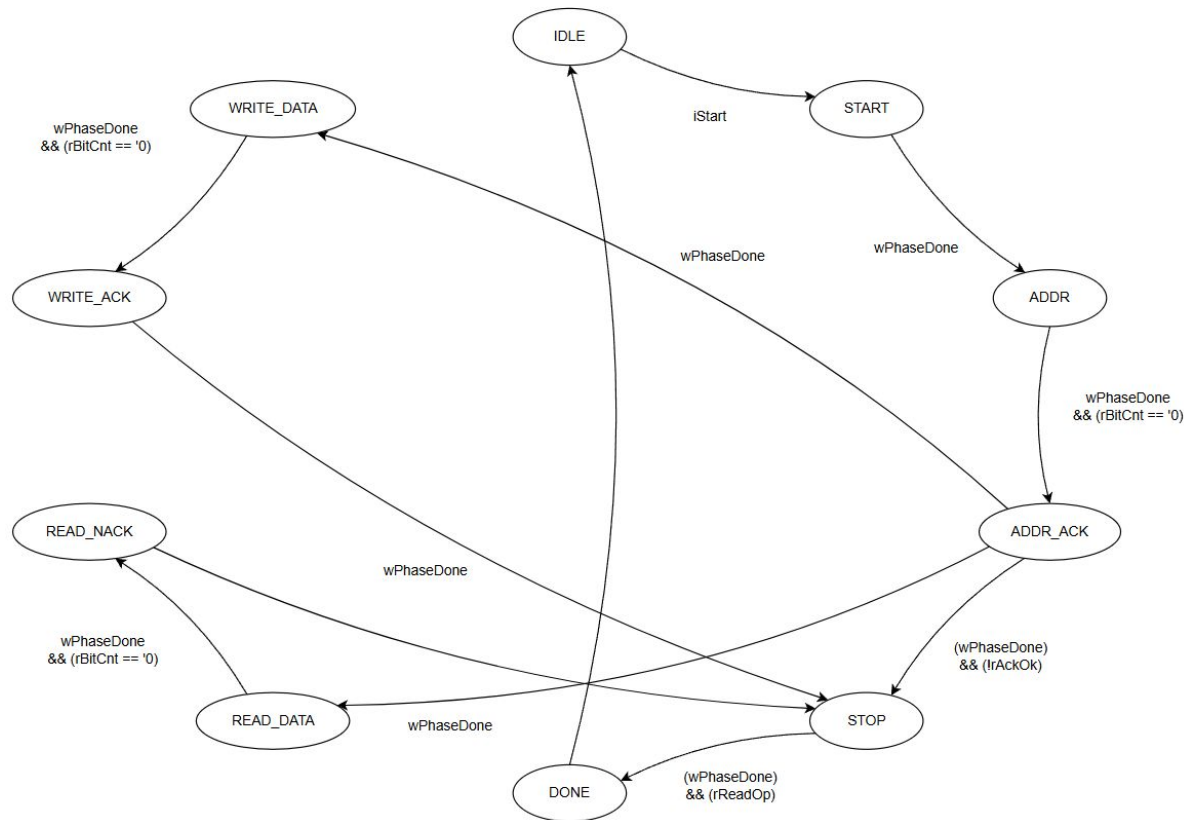
I2C 모듈 설계 주안점

- **상태 제어 복잡성:** START/STOP 컨디션, ACK/NACK 판별, 방향(Read/Write) 전환 등 고도화된 FSM 구현.
- **Open-Drain 구동:** 버스 충돌 방지를 위해 SDA 라인을 하드웨어적 Inout 포트 구성.

3.2 통신 모듈 설계: I2C_MASTER

Inter-Integrated Circuit Design

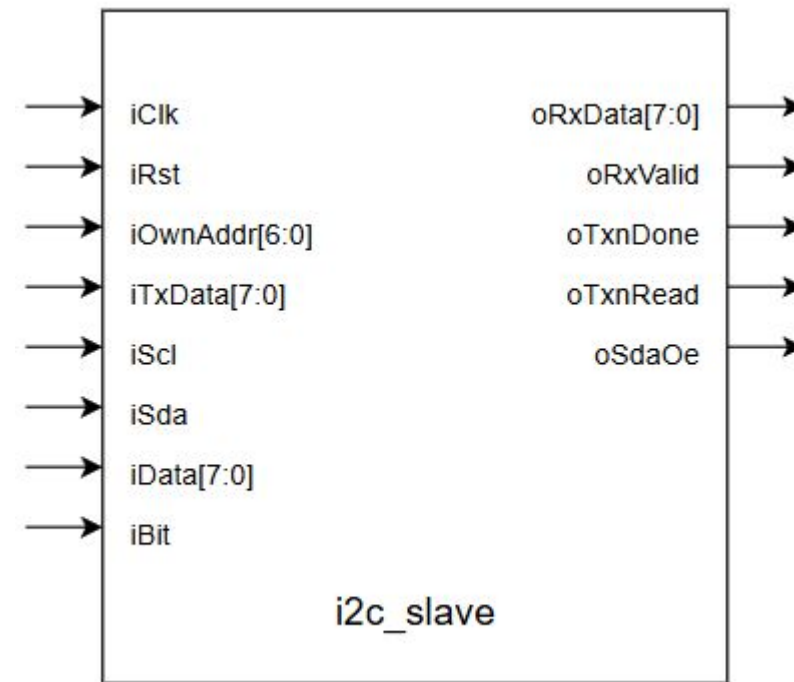
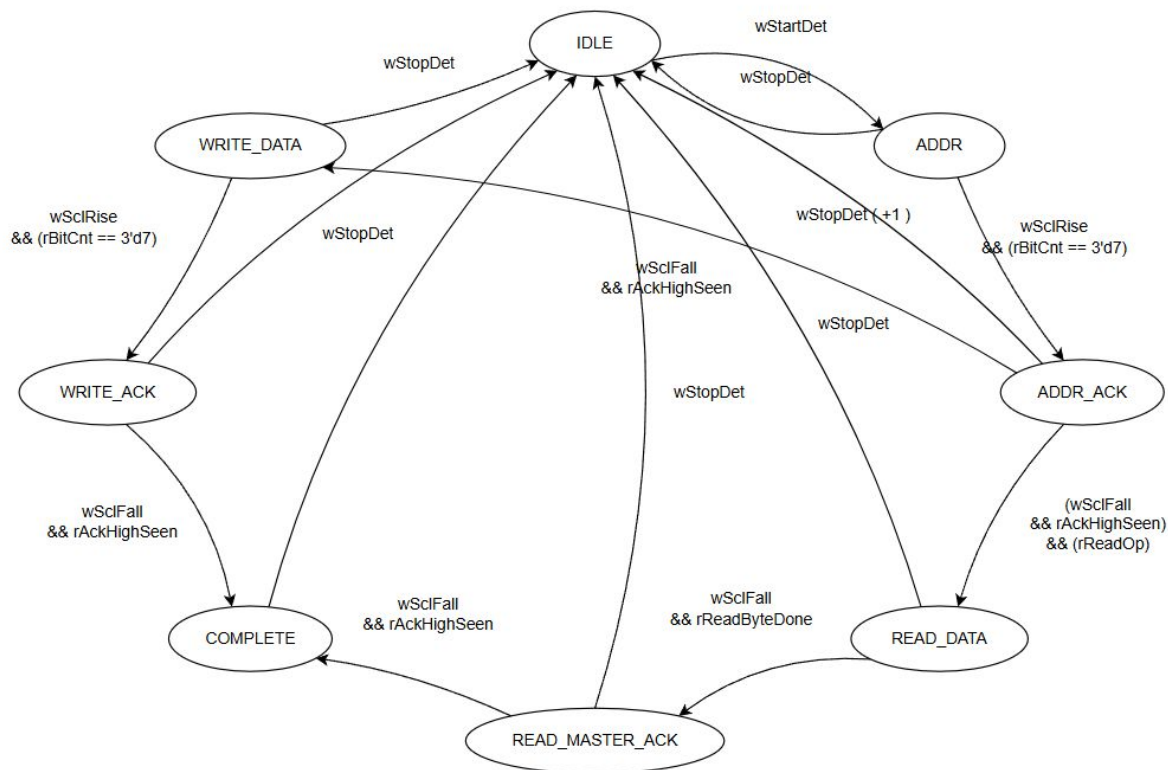
- **FSM 기반 제어:** State를 8개로 나누어 데이터 시트에 맞게 8bit data를 I2C통신 프로토콜로 변환
- **Master 신호:** Sclk과 Sda 신호를 만들어서 slave에게 전달한다



3.2 통신 모듈 설계: I2C_SLAVE

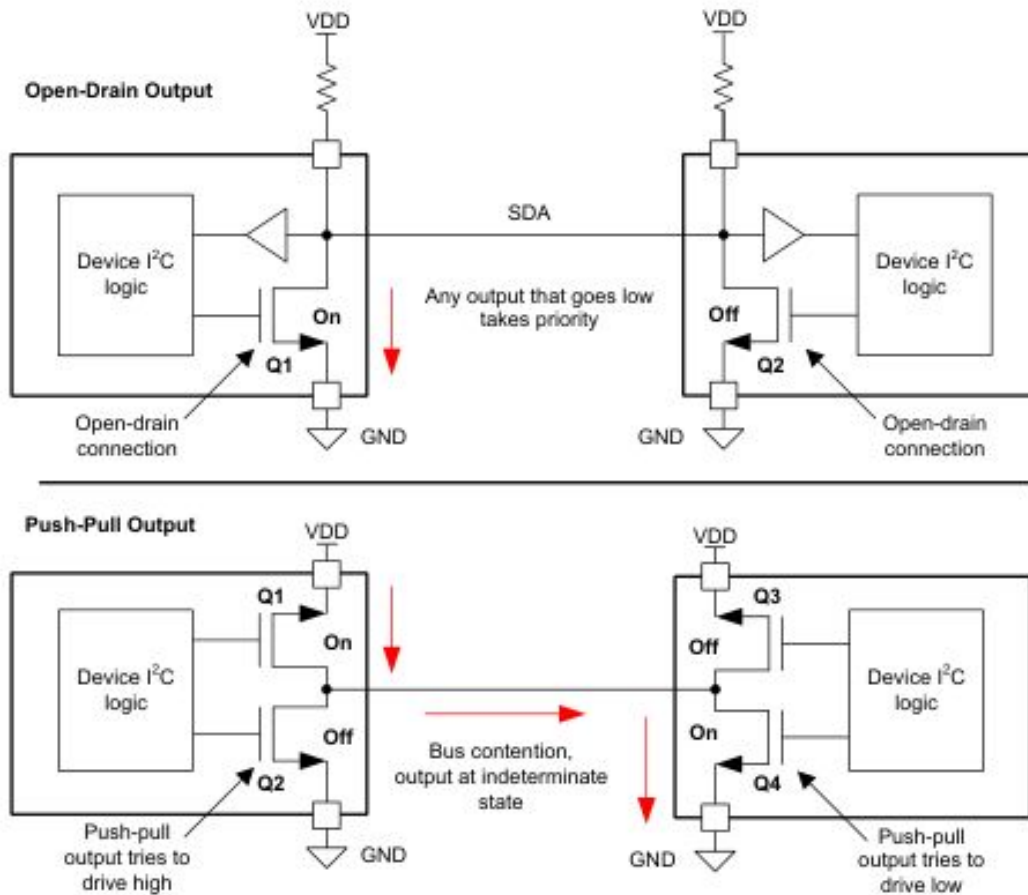
Inter-Integrated Circuit Design

- **FSM 기반 제어:** State를 5개로 나누어 데이터 시트에 맞게 8bit data를 spi통신 프로토콜로 변환
- **Slave 신호:** master의 scl과 sda 신호를 받아 8bit데이터를 수신하거나 sda신호선을 통해 데이터를 전송한다.



3.2 통신 모듈 설계: I2C Open Drain

Inter-Integrated Circuit Design

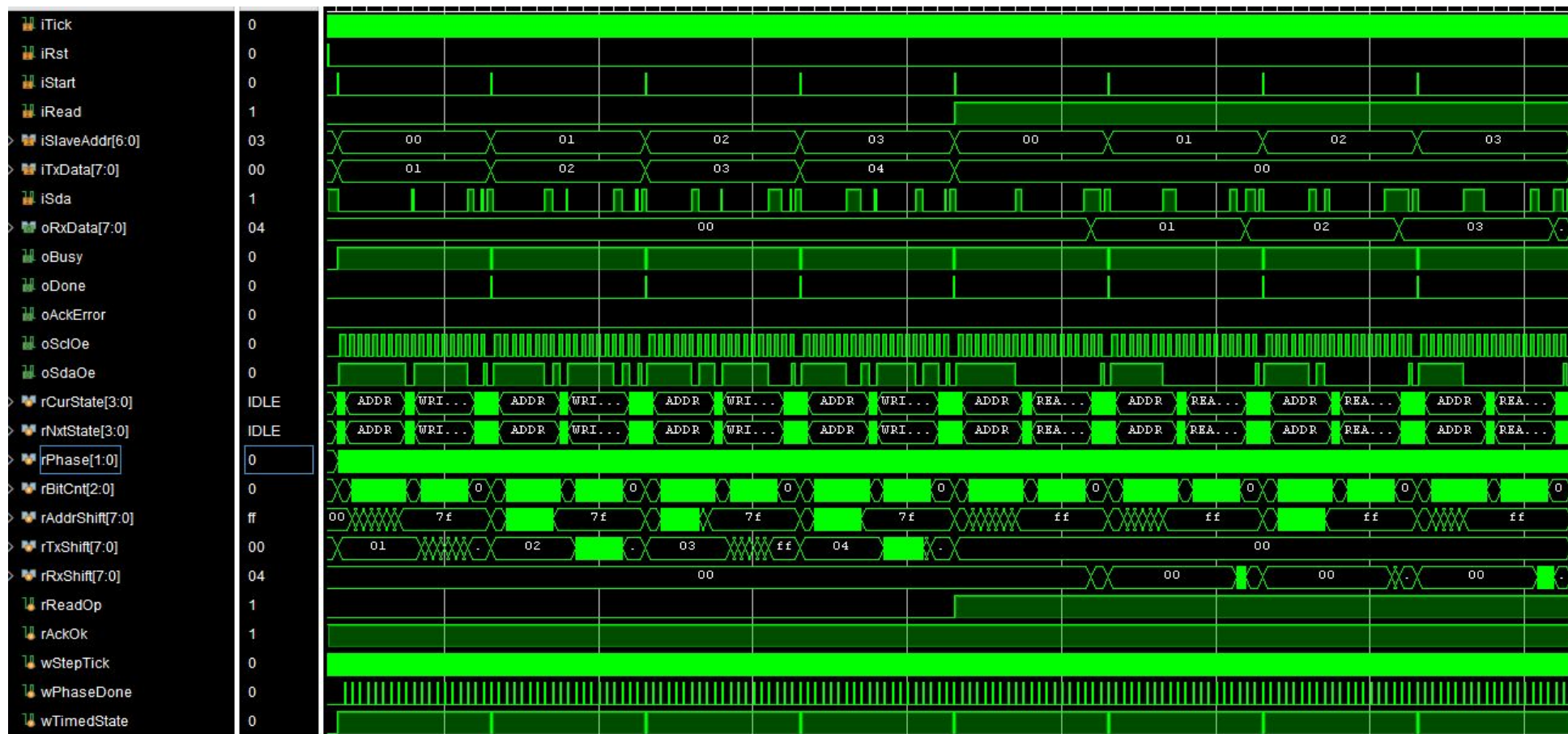


```
assign ioI2cScl = (wBusModeI2c && wI2cSclOe) ? 1'b0 : 1'bz;
```

```
assign ioI2cSda = (wBusModeI2c && wI2cSdaOe) ? 1'b0 : 1'bz;
```

3.2 Simulation: I2C_MASTER

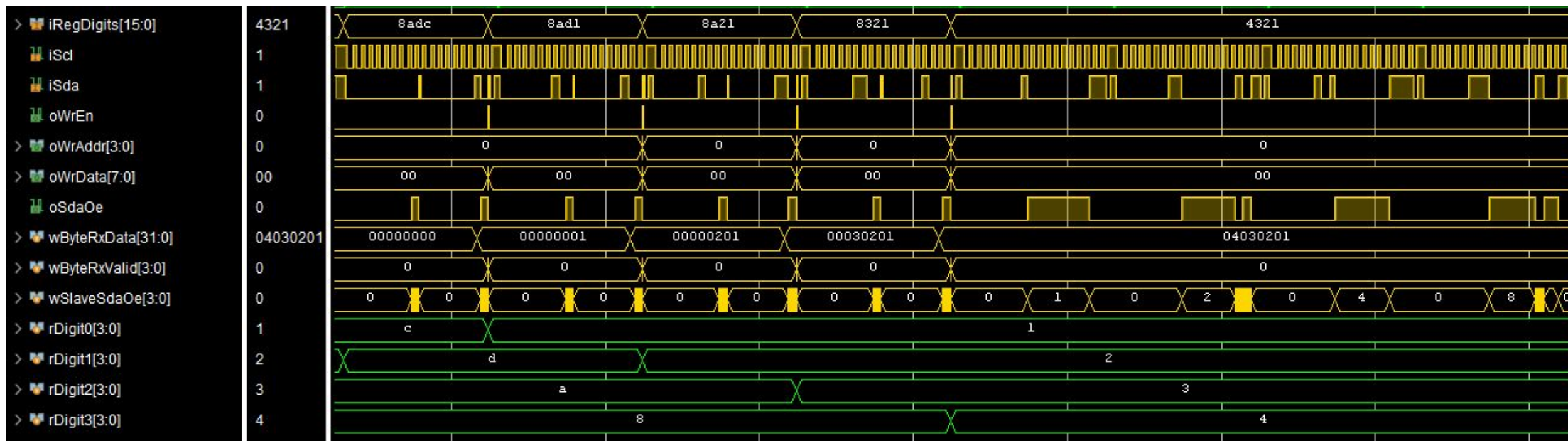
Waveform Analysis (기능 확인용 Testbench)



검증 포인트: I2C 시작/종료 확인, sda, scl 신호선 확인, 데이터 Write 이후 내부 **Register** 갱신 및 기대값 일치 여부 점검.

3.1 Simulation: I2C_SLAVE

Waveform Analysis (기능 확인용 Testbench)



검증 포인트: I2C 시작/종료 확인, 데이터 Write 이후 내부 **Register** 갱신 및 기대값 일치 여부 점검.

4. 검증 환경 확장: UVM 고도화

Universal Verification Methodology Strategy

기본 기능 확인을 넘어 객체지향 자동화 검증으로 진화

UVM 컴포넌트 구성

Agent, Driver, Monitor, Scoreboard 객체를 구성하여, 자극 (Stimulus) 생성과 결과 검증 과정을 완벽히 **객체지향적으로 자동화**합니다.

코너 케이스 스트레스 테스트

통신 Mode, Address 조합 및 쉬는 구간 없는 연속 전송 (**Back-to-back**) 환경을 강제 조성하여 코어의 한계를 테스트합니다.

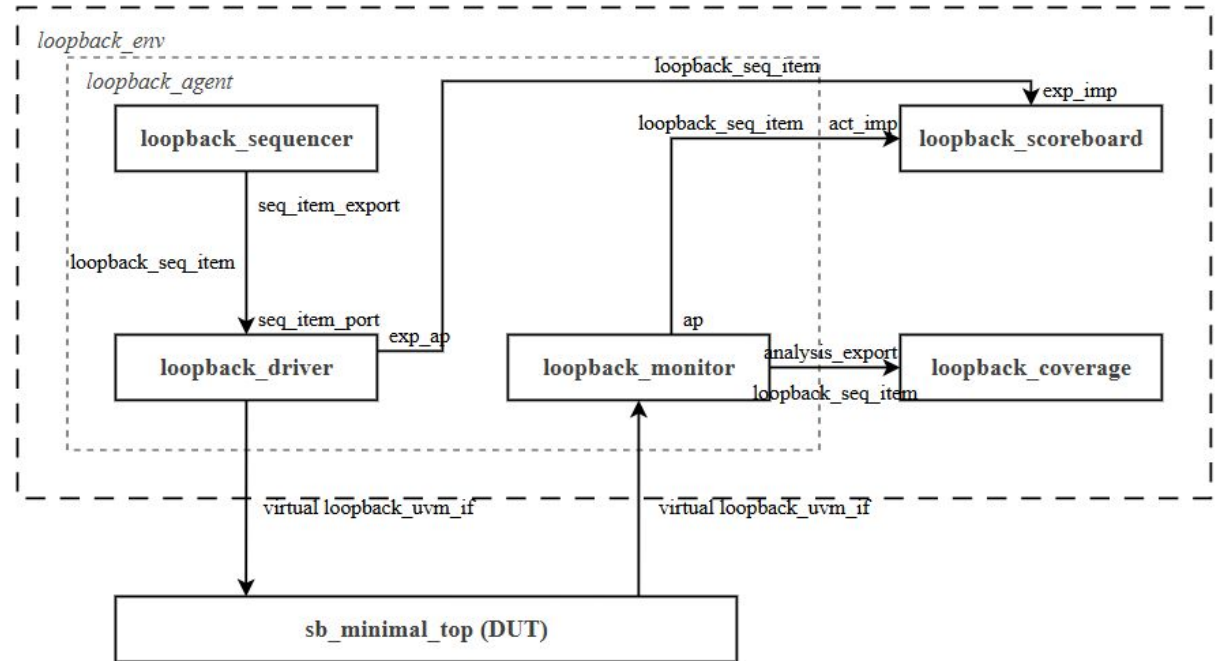
Error Injection & Coverage

고의적인 **ACK 누락(NACK)** 등 프로토콜 예외 상황을 주입하고, **커버리지(Coverage-driven)** 기반으로 누락된 테스트를 체계적으로 추적 보완합니다.

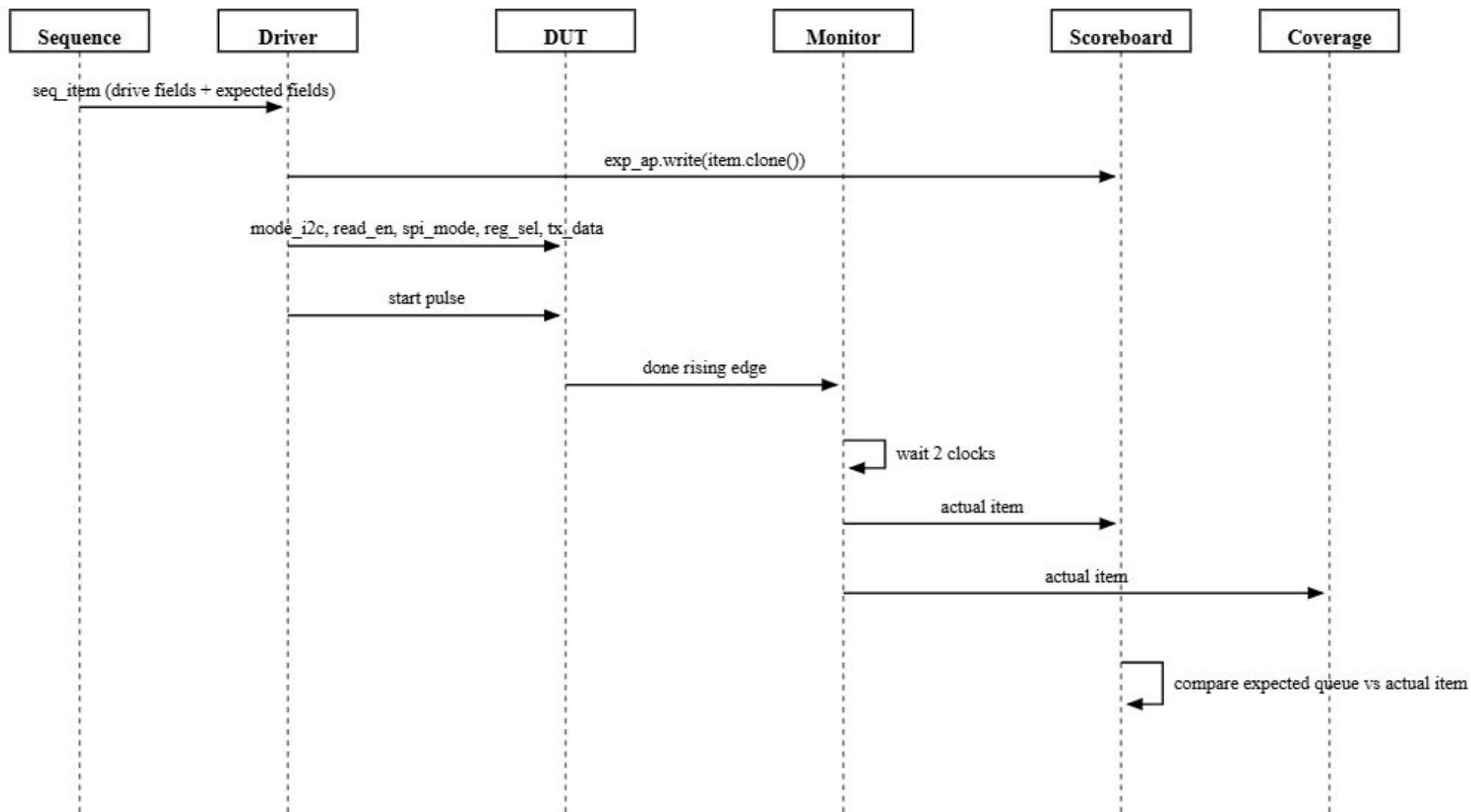
4. UVM 검증 환경 구조 (TLM Map)

Command & Observation Path

- **Sequence:** 트랜잭션을 생성하고 **reg_model**을 통해 Expected 결과를 산출합니다.
- **Driver:** DUT 입력 핀을 직접 구동함과 동시에 Expected Item을 Scoreboard로 전달(**exp_ap**)합니다.
- **DUT:** sb_minimal_top 내부 루프백 실행.
- **Monitor:** 완료 시점 기준 Actual 트랜잭션 복원 후 브로드캐스팅 (**ap**)합니다.
- **Scoreboard/Cov:** 비교 및 달성도 기록.



4. UVM 검증 환경 구조 (TLM Map)



4. UVM 시나리오

Universal Verification Methodology Strategy

1. Full 시나리오

SPI mode 0~3 및 reg0~3 전수 순회. Overwrite 동작과 I2C Write/Read를 통한 전체 기능 일괄 검증.

2.Random 시나리오

Protocol, R/W, Register, Data를 랜덤하게 흔들어 Directed Test가 놓칠 수 있는 순서 조합 64회 확인.

3. Negative 시나리오

반복 Overwrite, 급격한 Protocol Switching, 그리고 동작 중간 **Reset 주입(Abort)**을 통한 예외 상황 환경 파괴 검사.

4. Reset Recovery

Reset 이후 DUT 정상 동작 확인.

4. UVM Coverage

Universal Verification Methodology Strategy

1. 통신 모드 및 주소 조합 (Protocol & Addressing Combinations)

- cp_protocol / cp_spi_mode
SPI와 I2C의 프로토콜 커버 및 SPI 4가지 모드(CPOL, CPHA) 교차 검증.
- cp_reg_sel / cx_protocol_reg
모든 슬레이브 레지스터(0~3) 대상의 물리적 라우팅 및 개별 선택 검증
- cx_protocol_rw
데이터 방향(Read/Write) 검증.

2. 에러 주입 및 예외 스트레스 (Error Injection & Exceptions)

- cp_ack_error
I2C ACK/NACK 응답 경로를 관찰 대상으로 정의하고, 예외 응답 처리 가능성을 커버리지 항목으로 관리합니다.
- cp_protocol_switch
SPI 트랜잭션 도중 I2C로 전환되거나 그 반대의 상황에서 FSM이 안정적으로 대응하는지 검증.
- Ignore: Singleboard 루프백 환경에서는 슬레이브가 상시 존재하므로 NACK가 실제로 발생하지 않아, 해당 항목은 coverage에서 ignore_bins로 처리합니다.

4. UVM Coverage cont'd & Clocking Block Timing

Universal Verification Methodology Strategy

3. 코너 케이스 스트레스 (Corner Cases & Stress Tests)

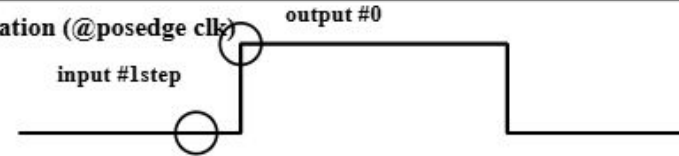
■ cp_overwrite / cx_overwrite_reg

쉬는 구간 없는 연속 전송(Back-to-back) 환경을 강제
조성하여, 동일 레지스터에 대한 연속 덮어쓰기 한계 테스트
수행.

■ cp_reset_event / cx_protocol_reset

물리적 하드웨어 리셋 직후, 각 프로토콜이 가장 첫 번째로
수행되는 복구 트랜잭션의 신뢰성 검증.

Timing Visualization (@posedge clk)



Clocking Block Declaration: drv_cb

default input #1step output #0;

[output #0].(Declared Output)

- rst	- spi_mode[1:0]
- start	- reg_sel[1:0]
- mode_i2c	- tx_data[7:0]
- read_en	

[input #1step].(Declared Input)

- rx_data[7:0]
- digits[15:0]
- busy
- done, ack_error

Clocking Block Declaration: mon_cb

default input #1step;

[input #1step].(Observation from Interface Signals)

- rst	- read_en	- tx_data[7:0]	- busy
- start	- spi_mode[1:0]	- rx_data[7:0]	- done
- mode_i2c	- reg_sel[1:0]	- digits[15:0]	- ack_error

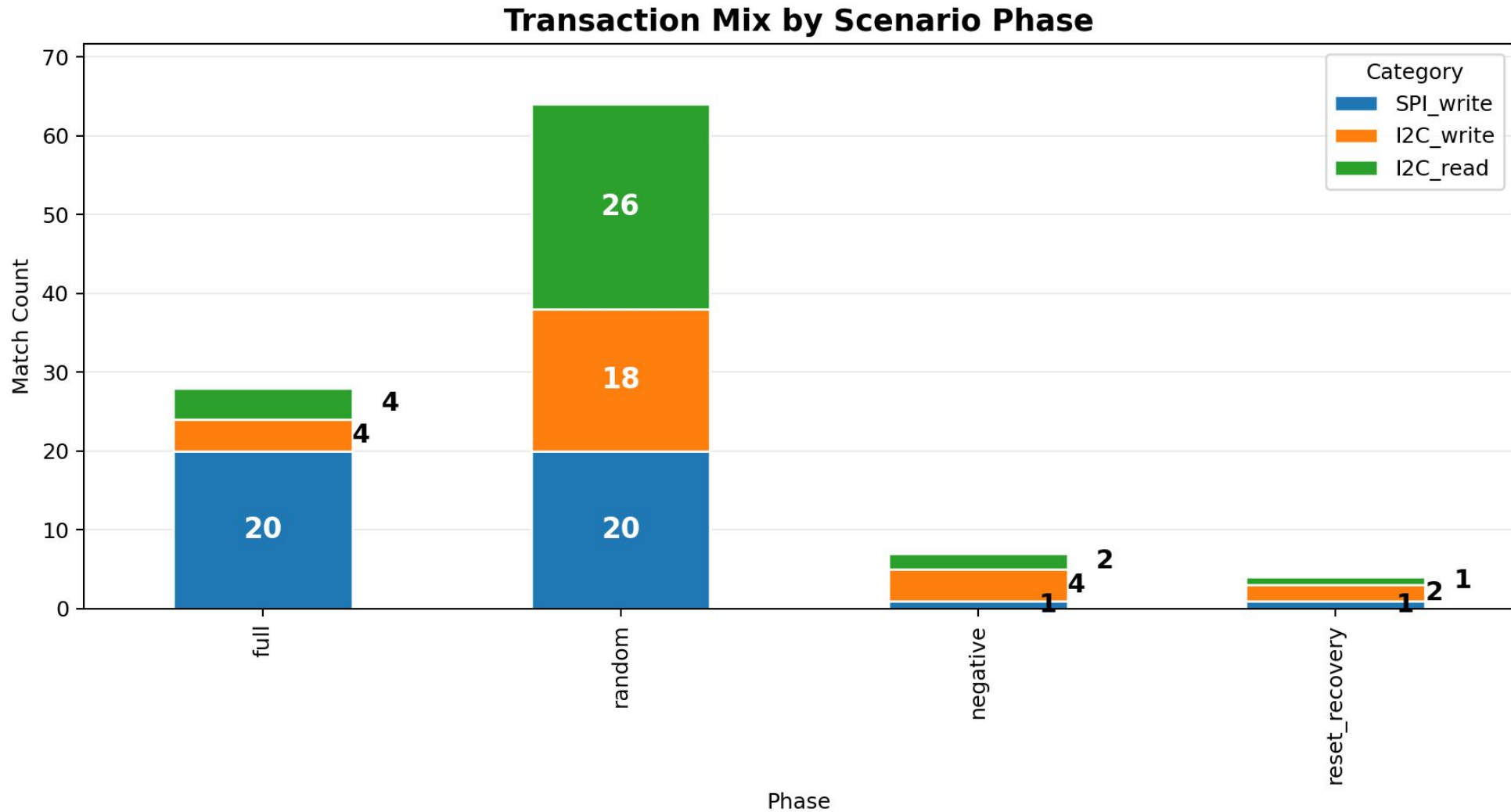
4. UVM Result

Universal Verification Methodology Strategy

CG	loopback_uvm_pkg::loop...		100.00%		62	0	62	0	100%	1	1	0	1	64	64
CP	cp_ack_error		100.00%		1	0	1	0	100%	1	1		1	0	
CP	cp_data_class		100.00%		4	0	4	0	100%	1	1		1	0	
CP	cp_overwrite		100.00%		2	0	2	0	100%	1	1		1	0	
CP	cp_protocol		100.00%		2	0	2	0	100%	1	1		1	0	
CP	cp_protocol_switch		100.00%		2	0	2	0	100%	1	1		1	0	
CP	cp_read_en		100.00%		2	0	2	0	100%	1	1		1	0	
CP	cp_reg_sel		100.00%		4	0	4	0	100%	1	1		1	0	
CP	cp_reset_event		100.00%		2	0	2	0	100%	1	1		1	0	
CP	cp_spi_mode		100.00%		4	0	4	0	100%	1	1		1	0	
CR	cx_overwrite_reg		100.00%		8	0	8	0	100%	1	1				0
CR	cx_protocol_reg		100.00%		8	0	8	0	100%	1	1				0
CR	cx_protocol_reset		100.00%		4	0	4	0	100%	1	1				0
CR	cx_protocol_rw		100.00%		3	0	3	0	100%	1	1				0
CR	cx_spi_mode_reg		100.00%		16	0	16	0	100%	1	1				0

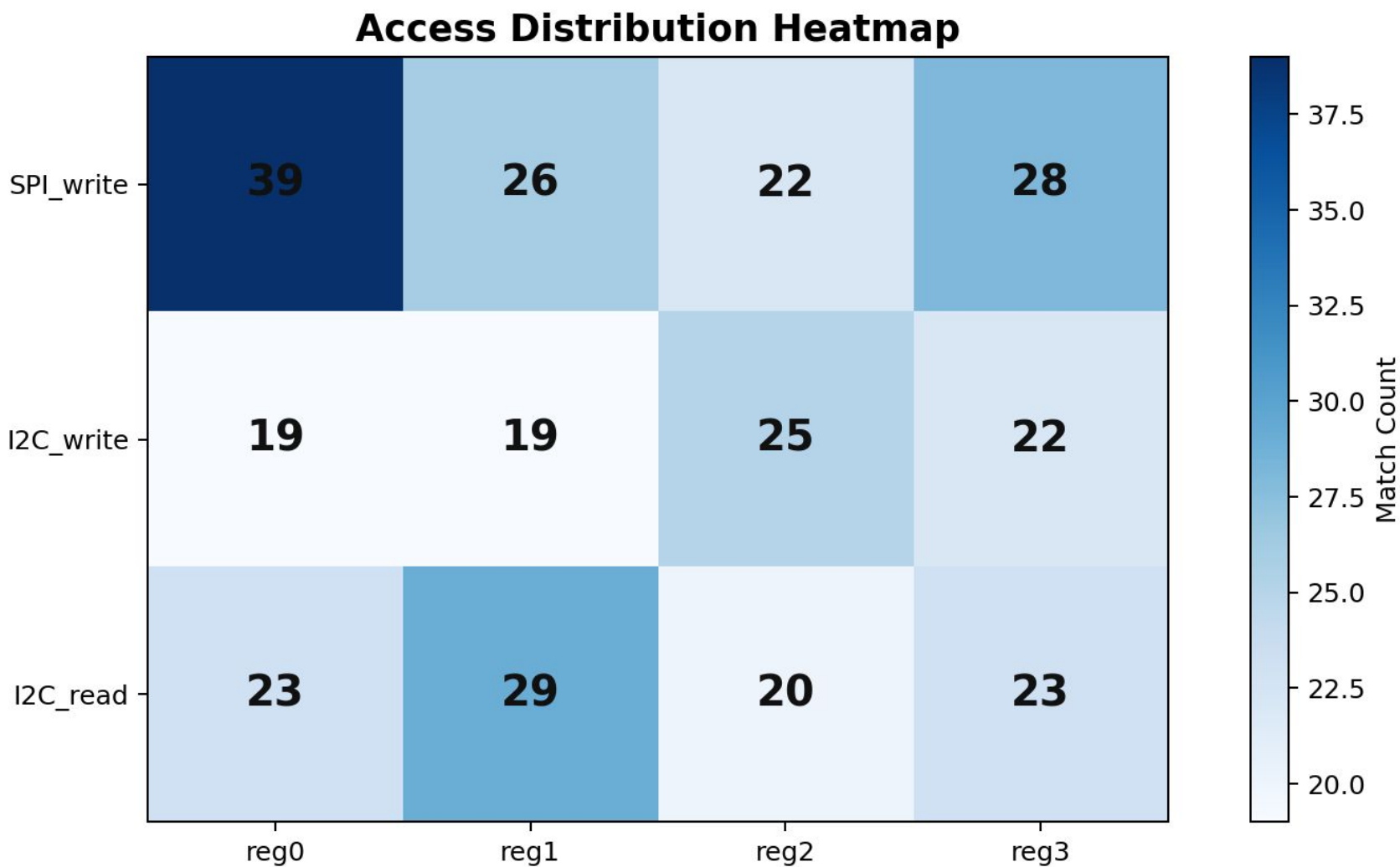
4. UVM Result

Universal Verification Methodology Strategy



4. UVM Result

Universal Verification Methodology Strategy



5.1 Trouble Shooting

Synchronization Issues in Hardware

실제 보드 연동 시 발생하는 통신 불량 문제

문제 상황 (Issue)

Singleboard의 내부 루프백은 정상이나, 실제 두 보드(Master-Slave)를 연결했을 때 간헐적으로 통신 실패 및 데이터 깨짐 현상 발생.

원인 및 해결 (Cause & Solution)

- **원인:** 외부 보드에서 입력되는 신호(SDA, MISO 등)가 **비동기 입력(Asynchronous Input)**으로 작용하여 메타스테빌리티 발생.
- **해결:** 모든 외부 통신 입력 신호 경로에 **2FF Synchronizer (2단 D-FF)**를 추가하여 CDC(Clock Domain Crossing) 문제를 해소.

5.2 결론 및 마무리

Conclusion

프로젝트 성과 요약

- **설계 완성:** SPI 및 I2C 직렬 통신 프로토콜을 RTL 레벨에서 직접 구현하여 칩/주소 선택 및 물리적 동작 원리를 깊이 체득하였습니다.
- **FPGA 검증:** 단일보드 검증을 넘어 Two-board 연결 환경에서의 하드웨어 타이밍(CDC) 문제 해결 및 실물 환경에서 동작을 검증했습니다.
- **UVM 검증:** 단순 기능 동작 확인(Waveform) 단계를 성공적으로 달성하고, UVM 기반 검증을 수행하여 rtl의 신뢰성을 높였습니다.

Q & A

경청해 주셔서 감사합니다.

서울기술교육 센터 | SPI/I2C 통신 프로젝트